



**Tecnológico
de Monterrey**

05. Procesamiento y visualización de datos

Ciencia de datos para la toma de decisiones II

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Programa de la sesión de hoy

- Terminar práctica de apertura de datos
- Repasar tidyverse y manejo de datos
- Repasar visualización en ggplot

Ejercicio de carga de datos

- 1. Vamos a terminar de cargar los datos de los archivos proporcionados el día de ayer.
- 2. Una vez cargada la información, explore el contenido y escriba en su script lo que contiene.

Procesamiento de datos

- * Es el proceso previo a la creación de análisis, modelos y visualizaciones.
- * Es un proceso a veces molesto, pero es necesario.
- * Consiste en tomar una base de datos y convertirla en otra que cumpla nuestras necesidades.
- * Siempre hay que tener en cuenta a qué queremos llegar.

Procesamiento de datos

Ejemplo de algunos procesos:

- 1) Limpieza de datos
 - 1) Convertir tipos de datos (texto -> numero)
 - 2) Eliminar registros con datos faltantes (NAs)
 - 3) Eliminar duplicados y outliers
 - 4) Estandarizar categorías (“CdMx” -> “CDMX”)
- 2) Unir tablas
- 3) Reestructurar tablas
- 4) Agregar datos por grupos
- 5) Cambiar nombres de columnas
- 6) Reclasificar grupos
- 7) Deflactar cifras
- 8) Transformar valores
- 9) Reconvertir columnas

¿Qué es el tidyverse?

El **tidyverse** es una colección de paquetes de R diseñados para su uso en ciencia de datos. Todos los paquetes miembros comparten una filosofía de diseño, estructura de datos y gramática comunes.

Para instalar el tidyverse completo, utilizamos la siguiente función:

```
install.packages("tidyverse")
```



@allison_horst

Paquetes del tidyverse



Paquetes del tidyverse



- La pipa es el elemento central del tidyverso y su misión es la de **concatenar funciones**. Opera vinculando los objetos del lado izquierdo con las funciones del lado derecho.
- Para escribirlo se utiliza:
 - **MAC**: command + shift + m
 - **Windows**: control + shift + m

Operador pipa (%>%)

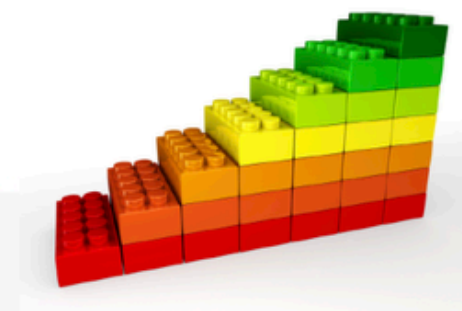


De manera coloquial, el operador pipa se debe leer como un **"y después"**. Por ejemplo, tomemos el objeto Juve.

* Con pipa:

```
# Un objeto Juve
Juve %>%
  despertarse(hora = "5:00 a.m.") %>% # Juve despierto (y despues...)
  levantarse(hora = "5:10 a.m.") %>% # Juve levantado (y despues...)
  bañarse() %>% # Juve despierto, levantado y bañado (y despues...)
  vestirse() %>% # Juve ahira vestido (y despues...)
  desayunar() %>% # Juve ahora desayunado (y despues...)
  salir_al_trabajo() # Juve rumbo al CIDE
```

Legos



Capas de cebolla



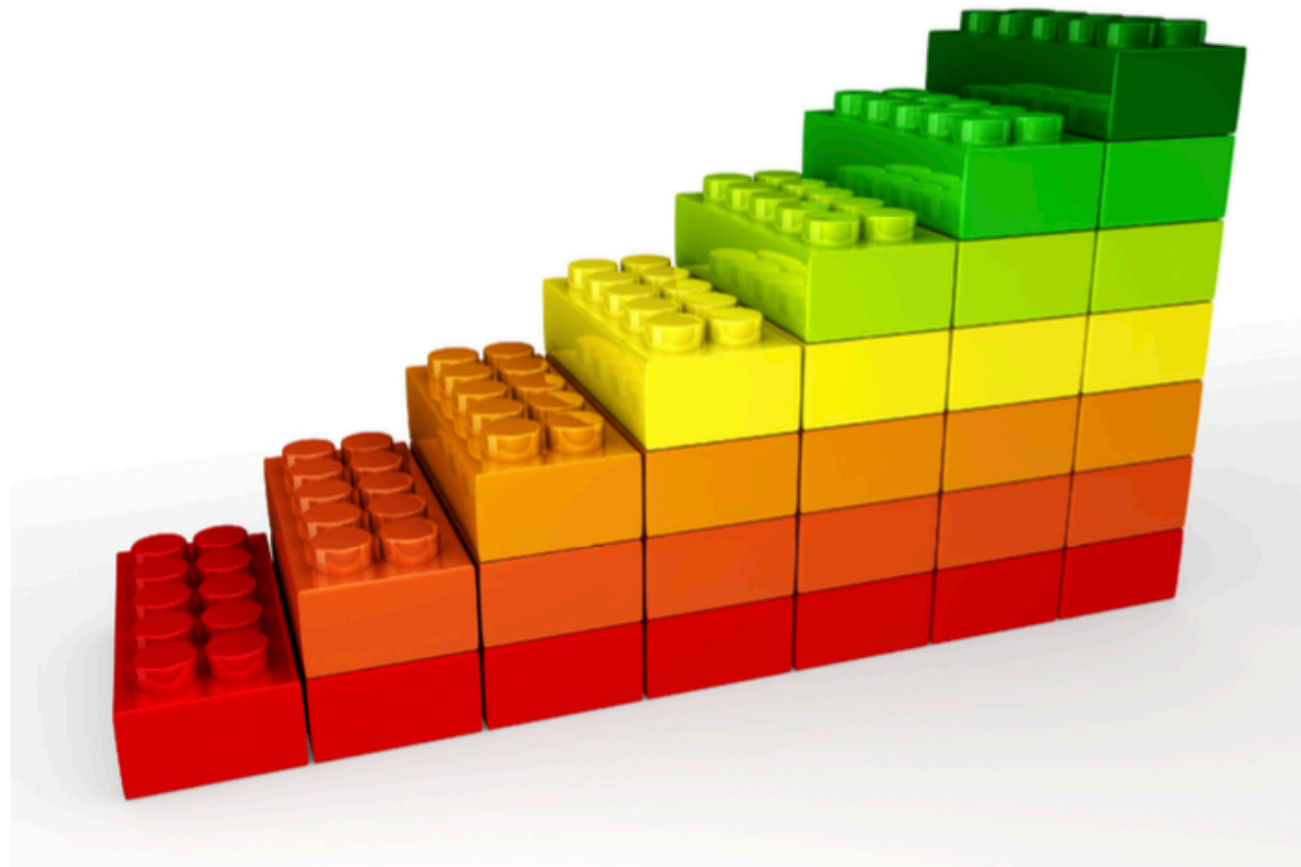
* Sin pipa:

```
# Sin la pipa
salir_al_trabajo(desayunar(vestirse(bañarse(levantarse(despertarse(Juve, hora = "5:00 a.m."), hora = "5:10 a.m.")))))
```

Operador pipa (%>%)



El operador %>% toma un objeto (en este caso Juve) y le va aplicando funciones (en este caso, todas las relativas a su rutina diaria matutina) de manera secuencial. Esto se va haciendo a manera de la unión de bloques de lego para armar una torre.



Operador pipa (%>%)



Tip 1 Si un día te sale un error y no sabes en que parte del pipeline se encuentra, corre una por una los bloques de código hasta que encuentres el error.

Tip 2 La estructura de trabajo con pipelines nos permite quitar (o silenciar, convirtiendo código en comentario) pedazos de código de manera muy fácil para hacer pruebas. Por ejemplo:

```
# Un objeto Juve
Juve %>%
  despertarse(hora = "8:00 a.m.") %>% # Juve despierto
  levantarse(hora = "8:10 a.m.") %>% # Juve levantado
#   bañarse() %>% # Juve despierto, levantado y bañado
  vestirse() %>% # Juve vestido
#   desayunar() %>% # Juve desayunado
  salir_al_trabajo() # Juve rumbo al CIDE
```


Operador pipa (%>%)



* Sin pipa:

```
# Declaramos el vector numérico `x`  
x <- c(0.109, 0.359, 0.63, 0.996, 0.515, 0.142, 0.017, 0.829, 0.907)
```

```
# Obtenemos el logaritmo de X, sacamos diferencias de un periodo, al resultado le  
sacamos la exponencial y después redondeamos el resultado.  
round(exp(diff(log(x))), 1)
```

```
## [1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

Capas de cebolla



* Con pipa:

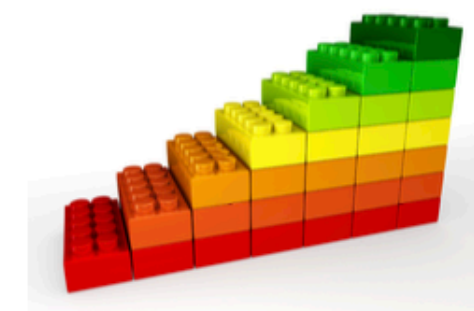
```
# La pipa ya viene importada en el tidyverse
```

```
# Hacemos las mismas operaciones que arriba:
```

```
x %>% log() %>%  
  diff() %>%  
  exp() %>%  
  round(1)
```

```
## [1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

Legos



Verbos del tidyverse



Las principales funciones del **tidyverse** para tratar con datos son las siguientes:

- **filter()**, para filtrar renglones,
- **select()**, para seleccionar columnas,
- **mutate()**, para generar nuevas columnas,
- **arrange()**, para ordenar los renglones en función de una variable,
- **group_by()**, para generar grupos o clusters de renglones,
- **summarise()**, para generar cálculos con las agrupaciones del **group_by()**

filter()

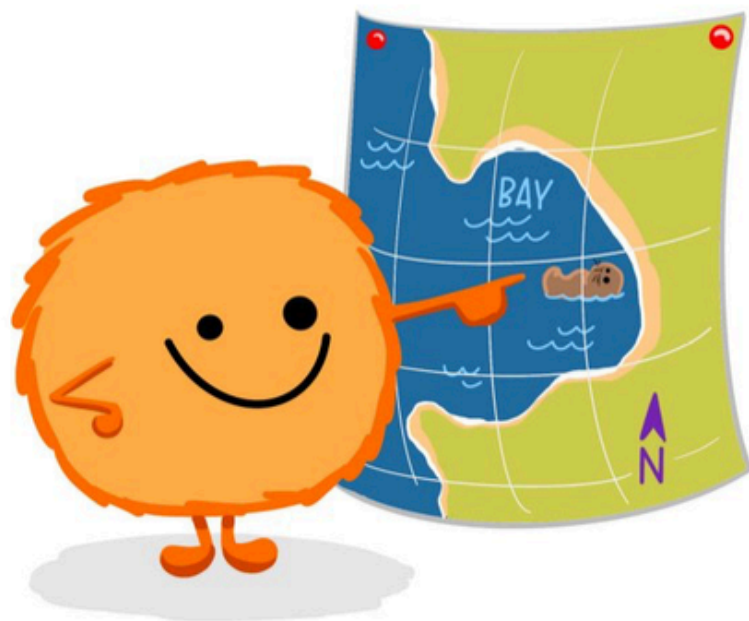


dplyr::filter()

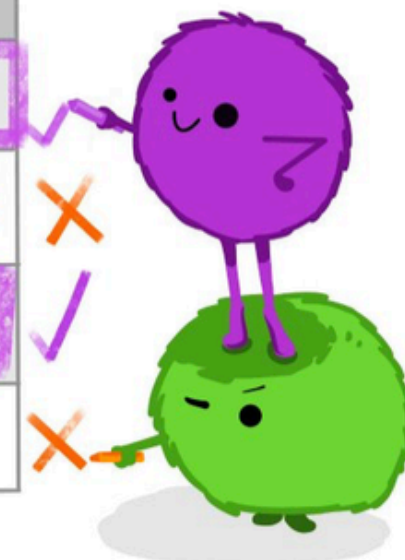
KEEP ROWS THAT
satisfy
your **CONDITIONS**

keep rows from... this data... ONLY IF... type is "otter" AND site is "bay"

```
filter(df, type == "otter" & site == "bay")
```



type	food	site
otter	urchin	bay
shark	seal	channel
otter	abalone	bay
otter	crab	wharf

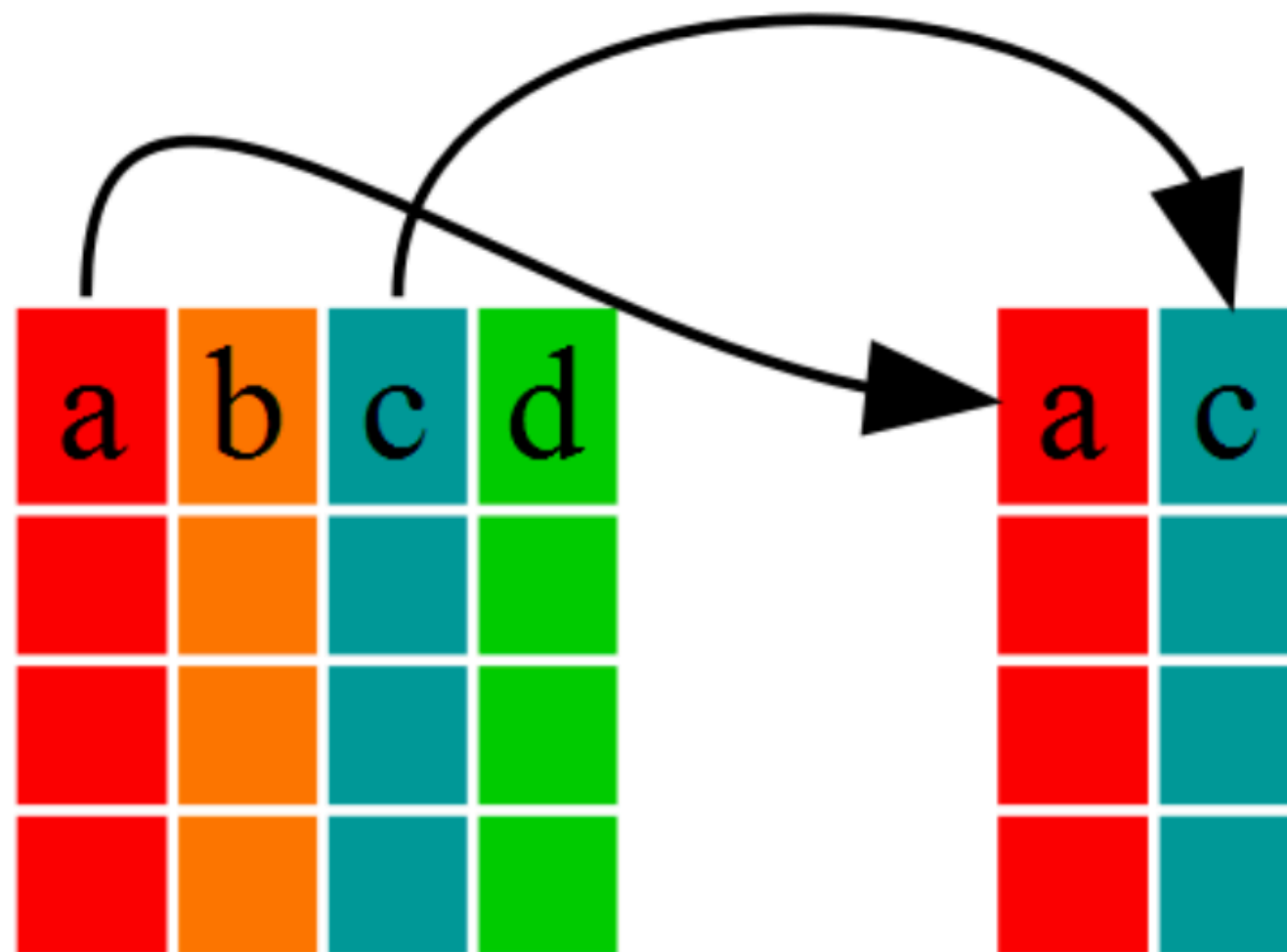


@allison_horst

select()



```
select(data.frame,a,c)
```



mutate()



arrange()



arrange()

storms

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21



storm	wind	pressure	date
Ana	40	1013	1997-07-01
Alex	45	1009	1998-07-30
Arthur	45	1010	1996-06-21
Arlene	50	1010	1999-06-13
Allison	65	1005	1995-06-04
Alberto	110	1007	2000-08-12

summarise()



city	particle size	amount ($\mu\text{g}/\text{m}^3$)
New York	large	23
New York	small	14



city	mean	sum	n
New York	18.5	37	2

London	large	22
London	small	16



London	19.0	38	2
--------	------	----	---

Beijing	large	121
Beijing	small	56



Beijing	88.5	177	2
---------	------	-----	---

Ejercicio de procesamiento de datos

- 1. Cargue los datos de pobreza que se encuentran en su carpeta del ejercicio bajo el nombre ***df_pob_ent2.xlsx***.
- 2. Utilizando los verbos del tidyverse vistos, responda las siguientes preguntas:
 - ¿Cuántas columnas tiene? ¿Cuántas filas?
 - ¿Qué representa cada fila en la tabla?
 - ¿Cuál es el estado con mayor porcentaje de población de pobreza (pobreza) en 2024?
 - ¿Cuál es el estado con menor porcentaje de carencia por acceso a la salud?

Ejercicio de procesamiento de datos

- ¿Cuáles son los 10 estados con mayor porcentaje de pobreza extrema (pobreza_e) en 2024?
- ¿Cuál es el promedio simple del porcentaje de pobreza moderada (pobreza_m) de los estados del centro del país (CDMX, Puebla, EDOMEX, Morelos e Hidalgo) en 2024?
- ¿En qué año alcanzó Chiapas su porcentaje más alto de población con carencia por acceso a la seguridad social?
- ¿Cuál es el valor más bajo de pobreza que ha alcanzado algún estado (con los datos de la tabla)?
- Obtenga el promedio y la desviación estándar del porcentaje de pobreza para todos los estados, para cada año

Ejercicio de procesamiento de datos

- Genere una gráfica de barras para visualizar el porcentaje de población en situación de pobreza para cada entidad para el año 2024. Que las barras de los 10 estados con mayor porcentaje aparezcan en un tono de rojo, los 10 estados con menor porcentaje aparezcan en un tono de verde, y el resto que aparezcan en alguna variante de amarillo.
- Genere una gráfica para visualizar la evolución de la variable de Pobreza Extrema (pobreza_e) para el estado de Chiapas a lo largo de todos los años disponibles en la tabla. Guarde la gráfica en un archivo *.png
- **Extra:** Genere un bucle (con un loop for) para generar las gráficas de todos los demás estados de la república.

Gracias :3